

Theorie Prüfung - Katharina Argarate

1)

Das Client-Server-Modell beschreibt die Zusammenarbeit zwischen einem Client (Browser) und einem Server (Webserver).

Der Browser fordert Daten an, und der Webserver stellt diese bereit und als Antwort zurücksendet.

Ablauf des HTTP-Request-Response-Zyklus:

1. Der Nutzer gibt die URL in die Adresszeile des Browsers ein.
2. Der Browser ermittelt über das DNS die IP-Adresse des Webserver (somit muss sich der Benutzer keine IP Adressen merken).
3. Der Browser baut dann eine Verbindung zum Webserver auf.
4. Der Browser sendet einen HTTP Request an den Server.
5. Der Webserver verarbeitet die Anfrage und sendet eine HTTP-Response zurück. Diese enthält unter anderem einen Statuscode und den HTML-Code der Webseite.
6. Der Browser empfängt die Antwort und verarbeitet und stellt das HTML dar. Währenddessen werden Verweise für alle Ressourcen (CSS Dateien, JS, Bilder, Schriftarten, etc.) gefunden.
7. Für jede dieser Ressourcen sendet der Browser einen eigenen Request an den Webserver und der Server liefert die Dateien zurück.
8. Sobald alle benötigten Ressourcen geladen sind, kann der Browser die Webseite vollständig (mit Layout, Bildern und Funktionen) darstellen.

Nach dem ersten Request sind aber weitere Requests notwendig, weil die erste HTML-Datei meist nur die Struktur der Webseite enthält.

Design, Skripte, Bilder, und andere Dateien müssen einzeln angefordert werden. So können Ressourcen wiederverwendet und vom Browser zwischengespeichert werden, was die Ladezeiten verbessert (und somit hat der Benutzer eine bessere Erfahrung!).

2)

HTTP-Requests bestehen aus:

- Startzeile: POST /api/login
- Header: Host, Content-Type, Authorization
- Body (Nutzlast): {"email":"user@test.de","password":"geheim"}

HTTP-Responses bestehen aus:

- Statuszeile: 401 Unauthorized
- Header: Content-Type, WWW-Authenticate
- Body (Nutzlast): {"error": "Token abgelaufen"}

Der Authorization-Header überträgt die Anmeldedaten bzw. das Zugriffstoken, damit der Server den Benutzer authentifizieren kann.

Der Statuscode 401 Unauthorized sagt uns, dass die Authentifizierung fehlgeschlagen ist oder das Token ungültig oder abgelaufen ist. Der Benutzer muss sich erneut anmelden oder ein neues Token verwenden, um weiterzukommen.

3)

```
<nav>
  <a href="https://www.shop.de">Startseite</a>
  <a href="/produkte/neu.html" target="_blank">Neue Produkte</a>
  <a href="#angebote">Zu den Angeboten</a>
</nav>
```

<https://www.shop.de> (Absolute URL) -> Hier öffnet der Browser die Webseite www.shop.de.

[./produkte/neu.html](/produkte/neu.html) (Relativer Pfad) -> Hier öffnet der Browser die Datei neu.html, was sich im Ordner "produkte" befindet, relativ zur aktuellen Seite.

<#angebote> (Fragment Anker) -> Hier springt der Browser zum Element mit der ID 'angebote' auf die gleiche Seite.

target="_blank" bewirkt, dass der Link in einem neuen Tab oder Fenster geöffnet wird.

Unterschiede:

Absolute URL: Enthält die vollständige Internetadresse (z. B. <https://www.shop.de>).

Relativer Pfad: Bezieht sich auf den Speicherort der aktuellen Webseite.

Fragment-Anker (#): Springt zu einem bestimmten Abschnitt innerhalb der gleichen Website.

4)

```
1 <header>
2 |   <div class="logo">NewsPortal</div>
3 </header>
4
5 <nav>
6 |   <a href="/">Home</a>
7 |   <a href="/politik">Politik</a>
8 </nav>
9
10 <main>
11 |   <article>
12 |     <h1>Wichtige Meldung</h1>
13 |     <p>Lorem ipsum dolor sit amet...</p>
14 |   </article>
15 </main>
16
17 <footer>
18 |   <p>© 2026 NewsPortal</p>
19 </footer>
20
```

Vorteile der semantischen Variante:

1. Screenreader können die Bereiche der Webseite erkennen und der Benutzer kann somit schneller zwischen Navigation, Inhalt und Footer wechseln.
2. Der HTML-Code ist besser verständlich und leichter wartbar, da die Bedeutung der Elemente direkt erkennbar ist.

Ein div ist trotzdem sinnvoll, in Fälle wo kein passendes semantisches Element vorhanden ist, zum Beispiel für reine Layout Gruppierungen oder Styling Zwecke ohne eigene Bedeutung.

5)

Breite:

- Content: 250px
- Padding links + rechts: 15px + 15px = 30px
- Border links + rechts: 3px + 3px = 6px
- Margin links + rechts: 25px + 25px = 50px

Gesamtbreite: $250\text{px} + 30\text{px} + 6\text{px} + 50\text{px} = 336\text{px}$

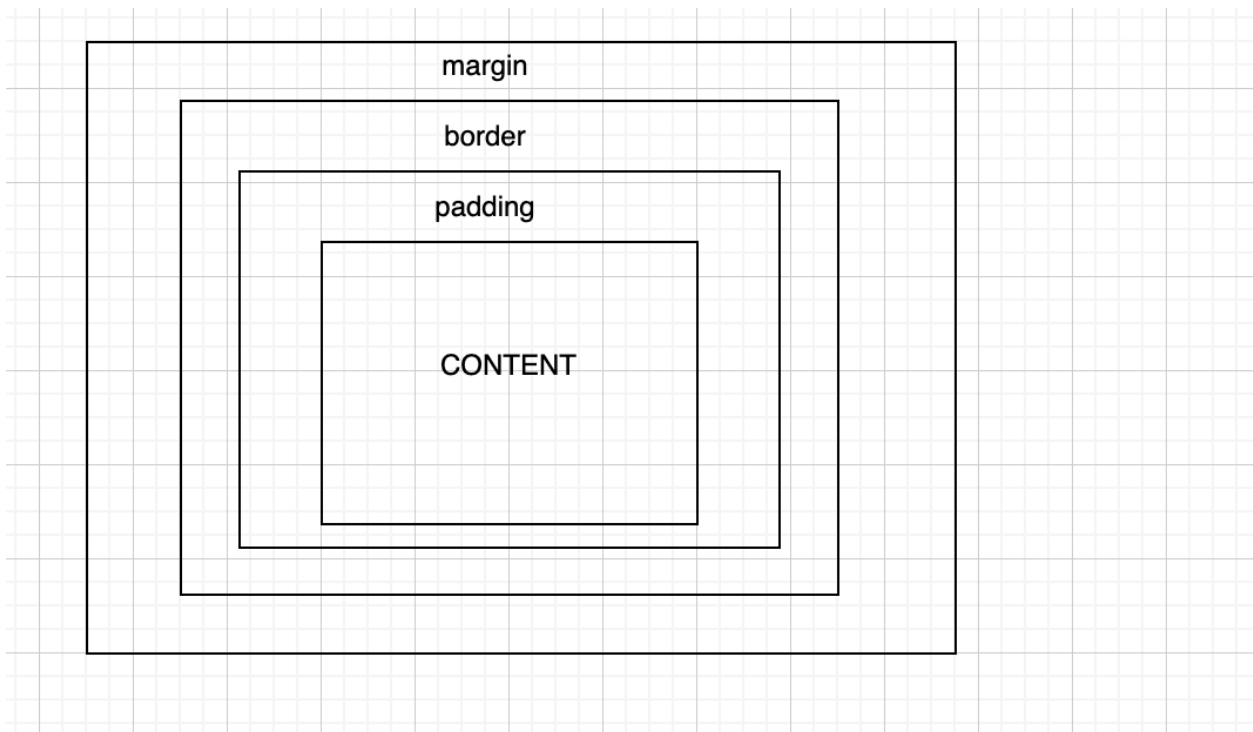
Höhe:

- Content: 150px
- Padding oben + unten: $15\text{px} + 15\text{px} = 30\text{px}$
- Border oben + unten: $3\text{px} + 3\text{px} = 6\text{px}$
- Margin oben + unten: $25\text{px} + 25\text{px} = 50\text{px}$

Gesamthöhe: $150\text{px} + 30\text{px} + 6\text{px} + 50\text{px} = 236\text{px}$

Insgesamt nimmt das Element $336\text{px} \times 236\text{px}$ Platz ein.

Skizze des Box-Modells:



margin = Abstand zu anderen Elementen

border = sichtbarer Rahmen

padding = Abstand zwischen Inhalt und Rahmen

content = eigentlicher Inhalt

6)

Zuordnung der Fehler:

- 404 Not Found - Client-Fehler
- 500 Internal Server Error - Server-Fehler

Unterschied:

- 4xx-Fehler: Die Anfrage des Clients ist fehlerhaft oder kann nicht verarbeitet werden (Problem auf Client Seite).
- 5xx-Fehler: Der Server hat ein Problem und kann eine gültige Anfrage nicht ausführen (Problem auf der Server Seite).

404-Fehler:

Die angeforderte Ressource (/api/products/99999) wurde nicht gefunden. Wahrscheinliche Ursache ist eine falsche Produkt-ID oder eine nicht vorhandene URL. Es muss vom Entwickler des Clients behoben werden, indem er z.B. die richtige ID sendet. Kann auch von die Person, die die API-Daten verwaltet behoben werden.

500-Fehler:

Der Server hat einen internen Fehler beim Erstellen der Bestellung (/api/orders). Mögliche Ursachen sind Programmierfehler, Datenbankprobleme, oder eine fehlerhafte Server-Logik. Beheben kann dies der Backend-Entwickler bzw. der Server-Administrator.

7)

URL Bestandteilen:

- **https://** -> Protokoll/Scheme (HTTPS für verschlüsselte Kommunikation)
- **www.onlineshop.de** -> Domain/Host (Adresse des Servers)
- **:8443** -> Port
- **/kategorien/elektronik** -> Pfad/Resource (Adresse der angeforderten Seite auf dem Server (kategorien wäre der Ordner)
- **?seite=2&sort=preis** -> Query-Parameter (zusätzliche Daten für die Anfrage)
 - **seite=2** -> Seite 2 anzeigen
 - **sort=preis** -> nach Preis sortieren
- **#ergebnisse** -> Fragment/Anker (springt zum Abschnitt der Seite, mit id="ergebnisse")

Protokoll -> Domain -> Port -> Pfad -> Query -> Fragment

8)

Unterschied zwischen HTTP und HTTPS:

HTTP überträgt Daten unverschlüsselt zwischen Browser und Server.

HTTPS verwendet eine verschlüsselte Verbindung (TLS), wodurch Daten geschützt und die Echtheit des Servers überprüft werden.

Warum wird das Bild blockiert?

Die Webseite wurde über HTTPS geladen, aber das Bild wird über unsicheres HTTP angefordert. Ein Angreifer könnte die Verbindung manipulieren und Inhalte verändern. Deshalb blockiert der Browser das Bild.

Konsequenzen bei einer Seite nur über HTTP:

Die gesamte Kommunikation wäre unverschlüsselt. Angreifer könnten Daten mitlesen oder verändern (z. B. Passwörter, Zahlungsinformationen, etc.). Die Identität des Servers kann auch nicht zuverlässig überprüft werden.

9)

Paragraph 1: Rot (#content p trifft zu)

Paragraph 2: Lila (Die letzte Regel ist gültig, obwohl eigentlich alle davor auch zutreffen, weil die letzte alle vorherigen überschreibt)

Paragraph 3: Cyan (Inline Styles haben die höchste Priorität)

Paragraph 4: Braun (diese Regel hat die höchste Spezifität (1 ID + 2 Klassen + 2 Elemente), also überschreibt sie die anderen Regeln)

10)

Die Boxen sind aktuell untereinander, weil div Elemente Block-Elemente sind, und daher die gesamte verfügbare Breite einnehmen und automatisch in einer neuen Zeile beginnen.

Zwei Ansätze (Grid + Inline Box):

Inline Block Version:

```
.container {
  width: 500px;
  border: 2px solid black;
  padding: 10px;
  text-align: center; /* zentriert die Boxen innerhalb des Containers */
}

.box {
  width: 100px;
  height: 100px;
  background: cornflowerblue;
  margin: 10px;
  display: inline-block; /* macht aus den Boxen Elemente, die nebeneinander stehen können*/
}
```

Flex Version:

```
.container {
  width: 500px;
  border: 2px solid black;
  padding: 10px;
  display: flex; /* ordnet die Boxen nebeneinander an */
  justify-content: center; /* zentriert die Boxen horizontal */
}

.box {
  width: 100px;
  height: 100px;
  background: cornflowerblue;
  margin: 10px;
}
```